Object-Oriented Programming

(3168-001)

Spring 2026

Object-Oriented Programming

Lecture 2: Python Basics I

Prof. Jaehyung Seo

(seojae777@konkuk.ac.kr)

Language Intelligence & Representation Lab.

What is Python?

Definition

Python is a high-level, interpreted, general-purpose programming language.

It emphasizes code readability through significant indentation and supports multiple programming paradigms:

- **Procedural**
- **Functional**
- **Object-Oriented**

Example Context

Python abstracts away memory management (C-style malloc) to let developers focus purely on logic and architecture.

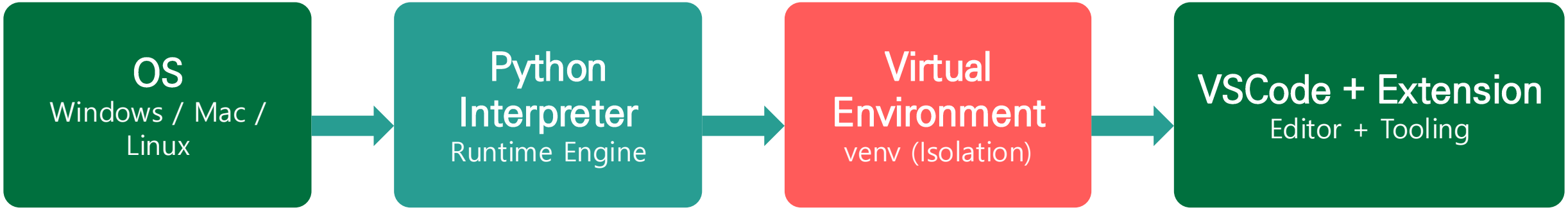
Instead of writing:

```
int* arr = malloc(sizeof(int)*n);
```

You simply write:

```
arr = [0] * n
```

Configuring the Workspace



Isolate your project dependencies using **Virtual Environments (venv)** and pair with **Python Extension** for code completion, linting, and debugging.

Hello, Object-Oriented World

A simple script demonstrating environment validation.

test_env.py

```python
import sys

def verify_environment():
    version = sys.version_info
    print(f"Python Version: {version.major}.{version.minor}")
    print(f"Executable Path: {sys.executable}")

if __name__ == "__main__":
    verify_environment()
```

Python as the Lingua Franca of AI

KEY TAKEAWAY

Python's rich ecosystem of object-oriented libraries makes it the undisputed standard for Artificial Intelligence and Machine Learning.

Frameworks like **TensorFlow**, **PyTorch**, and **Scikit-Learn** rely heavily on Python's OOP features to model complex neural networks and data pipelines elegantly.

Procedural vs. Object-Oriented

Feature	Procedural Programming	Object-Oriented Programming
Focus	Functions and Logic	Data and Behaviors (Objects)
State	Global or passed via parameters	Encapsulated within the object
Scalability	Hard to maintain as codebase grows	Highly modular, reusable, extensible

The Swiss Army Knife

Python does not force you into a single paradigm. You can mix all three seamlessly.

Procedural

Scripts, sequential logic

Quick file processing

Functional

map / filter / lambda

Math transformations

Object-Oriented

Classes and objects

Scalable applications

The Power of Indentation

BEFORE (C++ / Java Style)

```
if (x > 0) {
    do_something();
}
```

AFTER (Pythonic Style)

```
if x > 0:
    do_something()
```

Python uses **whitespace (indentation)** to define code blocks,
forcing developers to write visually clean code.

Variables and Dynamic Typing

Definition

Python is **dynamically typed**.

You do not declare variable types explicitly.

The interpreter infers the type at runtime based on the assigned value.

Int x = 42;

String name = "Hello";

Example

```
x = 42              # x is an int
print(type(x))      # <class 'int'>

x = "Hello"         # x is now a str
print(type(x))      # <class 'str'>
```

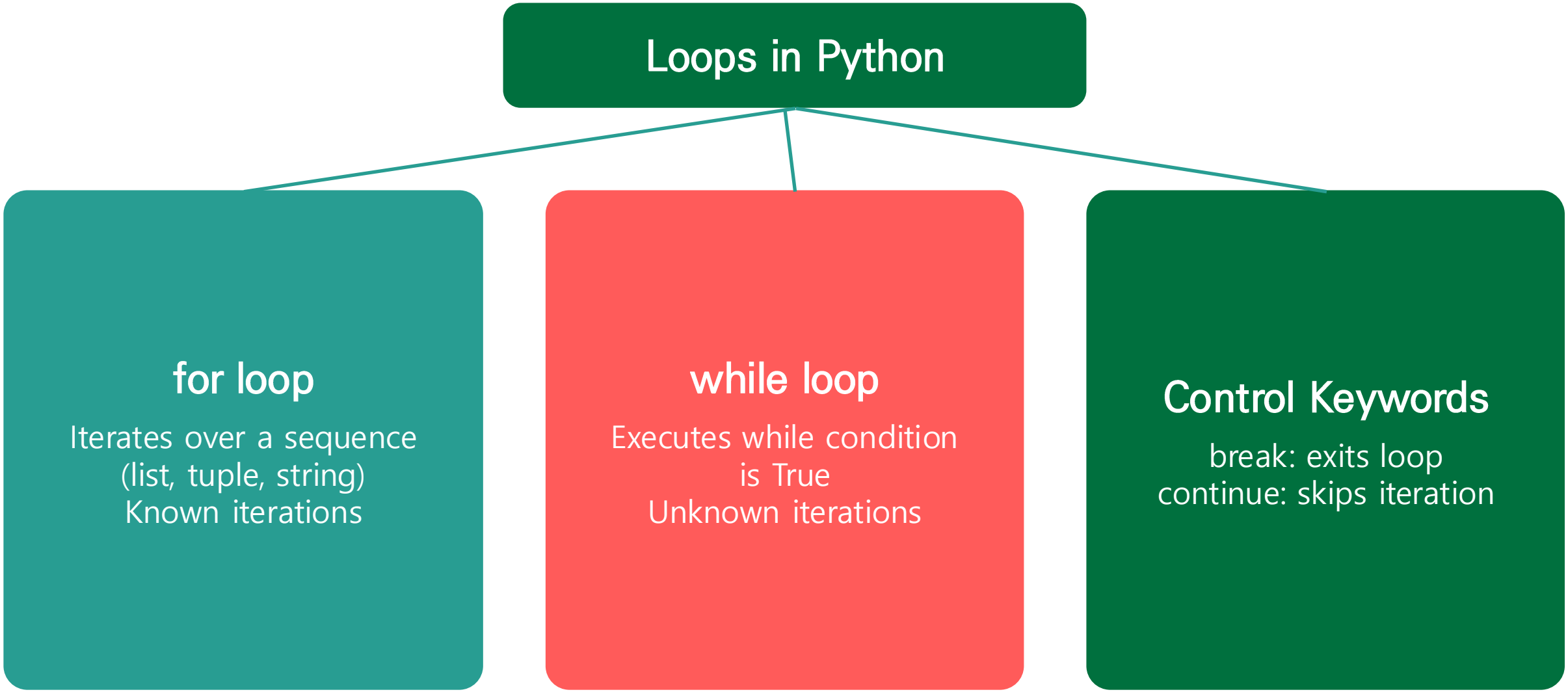
Control Flow (if / elif / else)

Demonstration of logical branching using boolean evaluation.

check_status.py

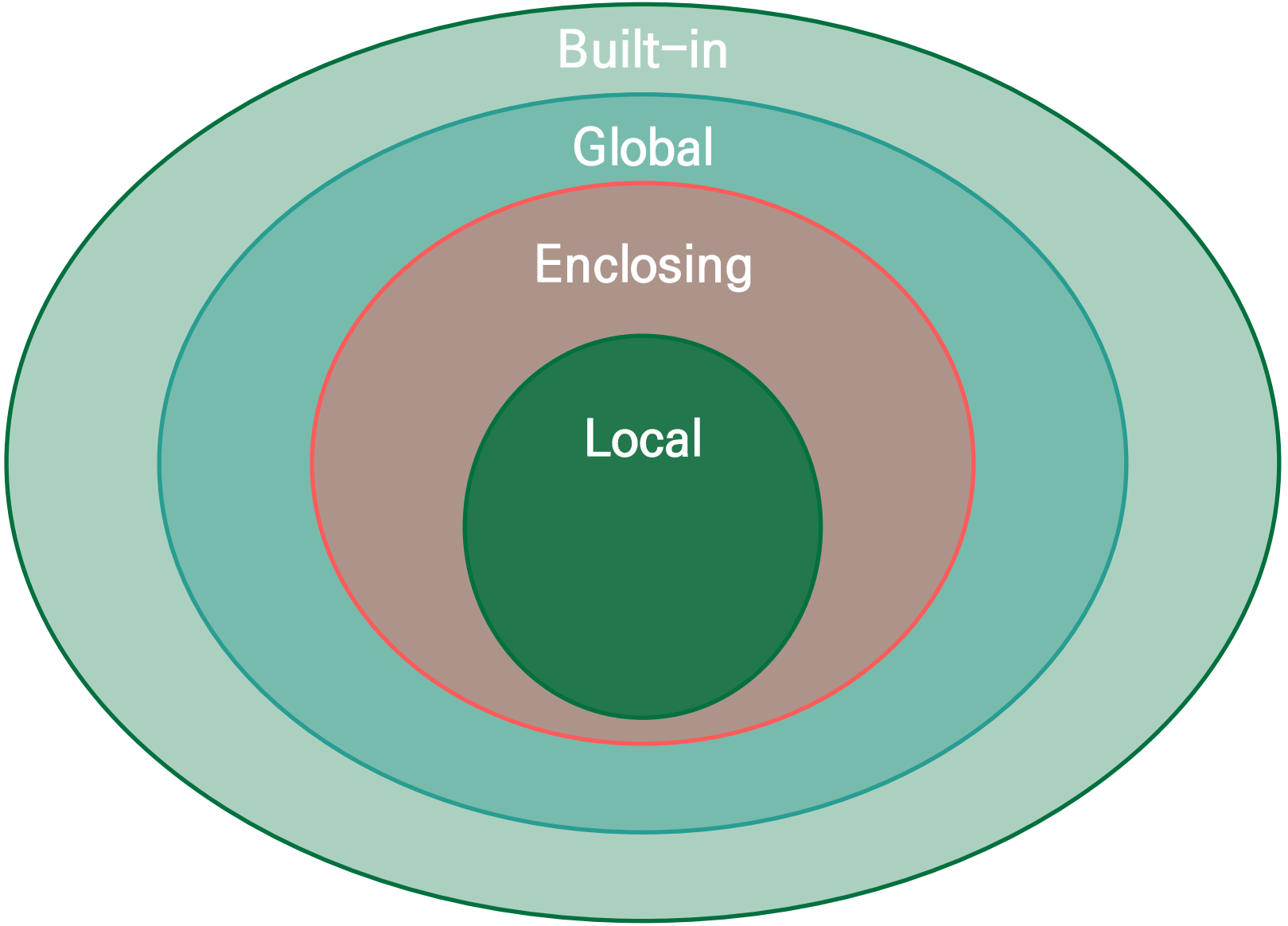
```
def check_status(is_active: bool, score: int) -> str:
    if not is_active:
        return "Account Disabled"
    elif score >= 90:
        return "Excellent"
    else:
        return "Needs Improvement"
```

Iteration and Loops



Functions and Scoping (LEGB)

- **L** Local (Inside function)
- **E** Enclosing (Nested function)
- **G** Global (Module level)
- **B** Built-in (Python builtins)



Error Handling

The Danger

Letting exceptions crash the program silently or unpredictably.

The Problem

Operations like dividing by zero or reading a missing file will raise Exceptions (ZeroDivisionError, FileNotFoundError), terminating the script.

The Solution

The try / except / finally block gracefully handles errors.

Core Data Structures

Structure	Syntax	Ordered?	Mutable?	Duplicates?
List	[5, 6]	Yes	Yes	Yes
Tuple	(1, 2)	Yes	No	Yes
Set	{1, 2}	No	Yes	No
Dictionary	{"a": 1}	Yes (3.7+)	Yes	Keys: No

Working with Lists

```
users = ["Alice", "Bob", "Charlie"]
users.append("Dave")      # Adds to the end

# Slicing: [start:stop:step]
top_two = users[0:2]    # ['Alice', 'Bob']
reversed_users = users[::-1]

# List Comprehension
lengths = [len(u) for u in users]
```

The Immutable Tuple

Definition

A Tuple is a sequence exactly like a list, but it is **Immutable**.

Once created, its contents cannot be altered, added to, or removed.

Example Context

Database connection coordinates (host, port) where you guarantee the application cannot accidentally overwrite the port number.

```
server_config = ("localhost", 8080)

# server_config[1] = 9000    # Raises TypeError!
```

Dictionaries (Key–Value Maps)

```
# Dictionary representing object-like state
student = {
    "name": "John",
    "major": "CS",
    "gpa": 3.8
}

# Safe access using .get()
age = student.get("age", 20)    # Returns 20 if key missing
student["graduated"] = False
```

The Power of Sets

KEY TAKEAWAY

Sets automatically enforce uniqueness and allow for lightning-fast mathematical set operations.

Remove duplicates from a list:

```
unique_items = set(my_list)
```

Find intersecting or differing elements between two datasets easily.

Mutability vs. Immutability

Mutable

(List / Dict)

A whiteboard where you can erase and rewrite text

Immutable

(Tuple / String)

A stone tablet with text carved into it

Understanding mutability is crucial for avoiding **side-effect bugs** in Object-Oriented design.

Everything is an Object

Definition

An **Object** is an encapsulation of:

- **Data** (State / Attributes)
- **Functions** (Behaviors / Methods)

bundled into a single, cohesive entity.

Example: Dog Object

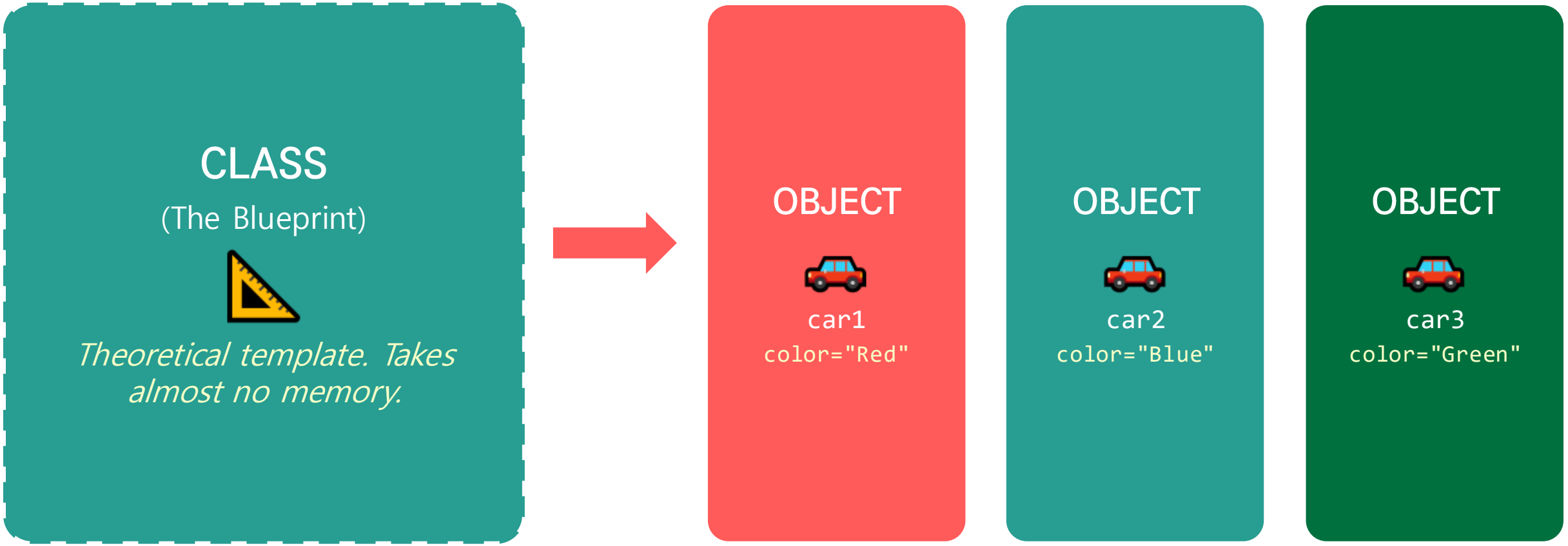
Data (Attributes):

- `name = "Buddy"`
- `age = 3`

Behaviors (Methods):

- `bark()`
- `eat()`

Classes vs. Objects



A **Class** is the theoretical blueprint. An **Object** is the physical realization in computer memory.

The __init__ Constructor

```
class Car:
    # The constructor method initializes the object
    def __init__(self, make: str, model: str):
        self.make = make    # Instance Attribute
        self.model = model  # Instance Attribute
        self.speed = 0


# Instantiating two separate objects
car1 = Car("Toyota", "Corolla")
car2 = Car("Ford", "Mustang")
```

The self Parameter

The Danger

Forgetting the self parameter in a method definition:

```
def start_engine():
```

The Problem

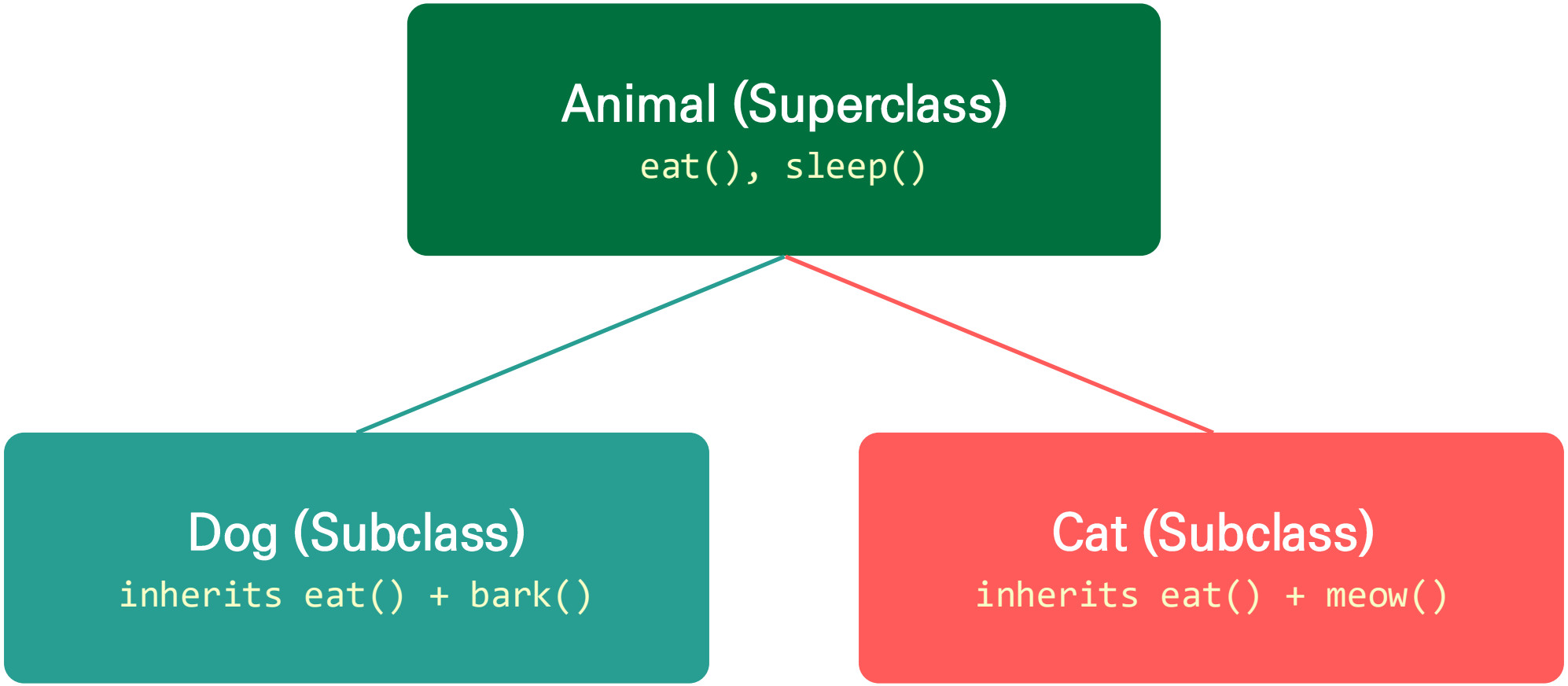
Python throws TypeError. When you call car1.start_engine(), Python secretly passes car1 as the first argument.

The Solution

Always include self as the first parameter:

```
def start_engine(self):
```

Inheritance (The "Is-A" Relationship)



Enforces the **DRY (Don't Repeat Yourself)** principle, reducing code duplication.

Polymorphism

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

def make_animal_speak(animal):
    # Polymorphism: we don't care what class it is
    print(animal.speak())
```

Encapsulation and Data Hiding

BEFORE (Public Access)

```
account.balance = -999

# Invalid state forced from outside!
```

AFTER (Encapsulated)

```
account.withdraw(999)

# Method validates rules first
```

Python uses **name mangling** (double underscores __)
to emulate private variables, protecting internal object state.

Instance vs. Class Attributes

Attribute Type	Defined Where?	Scope & Memory	Use Case
Instance	Inside __init__ using self	Unique to each object	name, speed, balance
Class	Inside the class body	Shared across ALL instances	Constants, total_count

Magic (Dunder) Methods

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    # Defines behavior for the '+' operator
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    # Defines string representation for print()
    def __str__(self):
        return f"Vector({self.x}, {self.y})"
```

Summary & Next Steps

Environment

- We secured a professional VSCode & Virtual Environment setup.

Foundation

- We reviewed Python's indentation-based syntax, dynamic typing, and control flow.

Data Structures

- We explored the time complexity and mutability of lists, tuples, dicts, and sets.

OOP Ready

- We understand Classes, Objects, self, Inheritance, Polymorphism, and Encapsulation.

Next Module

- Transitioning into Advanced Object-Oriented Design and Design Patterns.

Overview

From Design to Code
- Transitioning from abstract object-oriented design to writing actual Python code.

Type Hints & Checking
- Using Python's type hints to document data types and improve code safety.

Classes & Objects
- Learning the clean syntax for defining classes, adding attributes, and creating methods.

Initialization & Documentation
- Properly setting up an object's initial state and documenting code using docstrings.

Introducing Type Hints

Dynamic Typing
- Python is dynamically typed, meaning you do not have to declare variables.

Clarity and Intent
- Type hints are an optional feature that lets you specify what type of data (e.g., int, str) is expected.

Self-Documenting Code
- They act as built-in documentation, helping other developers understand how to use your functions and classes.

Type Hints in Practice

Code Snippet:

```
# A simple function with type hints
def greeting(name: str) -> str:
    return f"Hello, {name}!"

# Type hinting a variable
age: int = 25
```

Type Checking

Not Enforced at Runtime
- Python ignores type hints when the program is actually running.

Static Analysis
- Developers use external tools like mypy to check the code for type errors before running it.

Error Prevention
- This helps catch bugs early, such as trying to do math on a string instead of a number.

Running a Type Checker

Code Snippet:

```
# Save this in a file called test_type.py
def add_numbers(a: int, b: int) -> int:
    return a + b

# Intentional mistake: passing a string instead of an int
result = add_numbers(10, "20")

# Terminal command to check:
# $ mypy test_type.py
# Output: Argument 2 to "add_numbers" has
#   incompatible type "str"; expected "int"
```

Creating Python Classes

The class Keyword
- Python uses the class keyword to define a new object blueprint.

Clean Syntax
- Python's class syntax is famously minimal and requires very little setup code.

The pass Statement
- If a class has no content yet, we use the pass keyword to prevent syntax errors.

Defining a Basic Class

Code Snippet:

```
# Defining the simplest possible class
class MyFirstClass:
    pass

# Instantiating two separate objects from the class
a = MyFirstClass()
b = MyFirstClass()

print(a)
# Output: <__main__.MyFirstClass object at 0x...>
```

Adding Attributes

Object State

- Attributes are variables that belong to a specific object. They represent the object's state or data.

Dot Notation

- We access or assign attributes using dot notation (e.g., object.attribute).

Dynamic Addition

- In Python, you can dynamically attach new attributes to an object at any time.

Using Dot Notation

Code Snippet:

```
class Point:
    pass

# Create an object
p1 = Point()

# Dynamically adding attributes
p1.x = 5
p1.y = 4

print(f"Point coordinates: ({p1.x}, {p1.y})")
```

Making it Do Something

Methods as Behaviors
- Methods are simply functions that are defined inside a class.

Action-Oriented
- While attributes represent data (state), methods represent actions (behavior).

Encapsulation
- Methods allow the object to manage its own data internally, keeping related logic together.

Defining a Method

Code Snippet:

```
class Point:
    # A method definition
     def reset(self):
         self.x = 0
         self.y = 0

p = Point()
p.reset()     # Calling the method
print(p.x, p.y)  # Output: 0 0
```

Talking to Yourself (self)

The self Parameter
- Every instance method must have self as its first parameter.

Implicit Passing
- When you call a method on an object, Python automatically passes the object itself as the first argument.

Accessing State
- self is the key that allows a method to read or modify the specific object's attributes.

How self Works

Code Snippet:

```
class Point:
    def reset(self):
        self.x = 0
        self.y = 0

p = Point()

# Standard way (Python passes 'p' to 'self')
p.reset()

# What happens under the hood (explicit call)
Point.reset(p)
```

More Arguments

Multiple Parameters
- Methods can accept any number of arguments, just like regular functions.

Order Matters
- self must always be the very first parameter in the definition.

Passing Values
- When calling the method, you provide values for all arguments except self.

Passing Arguments to Methods

Code Snippet:

```
import math

class Point:
    def move(self, x: float, y: float) -> None:
        self.x = x
        self.y = y

p1 = Point()
# We skip 'self' and pass the two float arguments
p1.move(5.0, 8.0)
print(p1.x)  # Output: 5.0
```

Initializing the Object

The Constructor
- Python uses a special method called __init__ to initialize objects.

Automatic Execution
- __init__ is called automatically the moment an object is created.

Initial State
- It is the perfect place to set up the default attributes an object needs to function properly.

Basic Initialization

Code Snippet:

```
class Point:
    # __init__ runs automatically upon creation
     def __init__(self, x: float, y: float):
         self.x = x
         self.y = y

# Pass initialization values directly to the class
p = Point(3.5, 4.0)
print(f"Point created at {p.x}, {p.y}")
```

Why Initialization Matters

Data Integrity
- Guarantees that objects always start in a valid, usable state.

Preventing Errors
- Stops AttributeError crashes that happen when a method tries to use an attribute that hasn't been created yet.

Cleaner Code
- Removes the need to scatter attribute assignments throughout your main script.

Initialization vs Manual Assignment

Code Snippet:

```
# Bad Practice: No __init__
class BadPoint:
    pass
bp = BadPoint()
# bp.move(5, 5)  # CRASH! bp.x doesn't exist

# Good Practice: With __init__
class GoodPoint:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

gp = GoodPoint(0, 0)  # Safe! State is guaranteed.
```

Type Hints and Defaults

Default Arguments
- Python allows you to assign default values to parameters using the = sign.

Optional Inputs
- This makes object creation easier, as users only need to provide arguments if they want to change the defaults.

Combining with Hints
- Type hints and default values work together seamlessly in the method definition.

Using Defaults in __init__

Code Snippet:

```
class Point:
    # Type hints and default values combined
     def __init__(self, x: float = 0.0, y: float = 0.0):
         self.x = x
         self.y = y

# Uses defaults: x=0.0, y=0.0
p1 = Point()
# Overrides defaults: x=5.0, y=4.0
p2 = Point(5.0, 4.0)
# Overrides just y: x=0.0, y=8.0
p3 = Point(y=8.0)
```

Explaining Yourself with Docstrings

Code Documentation
- Docstrings are string literals used to explain what a module, class, or method does.

Triple Quotes
- They are enclosed in triple quotes (""") and can span multiple lines.

Accessibility
- External tools and Python's built-in help() function read these docstrings to assist other developers.

Class-Level Docstrings

Code Snippet:

```
class Point:
    """
    Represents a point in two-dimensional
    geometric coordinates.

    This class allows for initialization,
    movement, and distance calculations
    between points.
    """
    def __init__(self, x: float = 0.0, y: float = 0.0):
        self.x = x
        self.y = y
```

Method-Level Docstrings

Code Snippet:

```
class Point:
    # ... (init omitted) ...

    def move(self, x: float, y: float) -> None:
        """
        Move the point to a new location
        in 2D space.

        :param x: The new x-coordinate.
        :param y: The new y-coordinate.
        """
        self.x = x
        self.y = y
```

Modules and Packages

Modules as Files

- In Python, a module is simply a .py file containing Python code (classes, functions, variables).

Packages as Directories

- A package is a directory containing one or more Python modules, allowing for a hierarchical structure.

Namespaces

- Modules and packages help organize code and prevent name collisions by creating separate namespaces.

Importing from Modules (Code)

Code Snippet:

```
# Assuming a file named 'database.py' exists in the same folder
# inside database.py:
# class Database:
#     pass

# In our main script:
import database

# Accessing the class requires the module name prefix
db = database.Database()
```

Organizing the Modules

Targeted Imports

- Instead of importing a whole module, you can import specific classes directly into your current namespace.

The from ... import ... Syntax

- This syntax makes code cleaner by removing the need to type the module name repeatedly.

Flat is Better Than Nested

- While packages can be deeply nested, Python philosophy prefers shallow, "flat" package structures.

Using from ... import (Code)

Code Snippet:

```
# importing just the class we need
from database import Database
from api.network import Server

# Now we don't need the module prefix
db = Database()
server = Server()
```

Absolute Import Syntax (Code)

Code Snippet:

```
# Project structure:
# my_project/
# ├── ecommerce/
# │   ├── payments.py
# │   └── shopping_cart.py

# Inside shopping_cart.py
from ecommerce.payments import process_credit_card

process_credit_card()
```

Absolute import Syntax (Code)

Code Snippet:

```
# Project structure:
# my_project/
# ├── ecommerce/
# │   ├── payments.py
# │   └── shopping_cart.py

# Inside shopping_cart.py
from ecommerce.payments import process_credit_card

process_credit_card()
```

Relative imports

Current Location
- Relative imports specify the path relative to the current module's location.

Dot Syntax
- They use a single dot (.) for the current package, and double dots (..) for the parent package.

Refactoring
- They can be useful if you plan to rename your top-level package, as the internal links won't break.

Relative import Syntax (Code)

Code Snippet:

```
# Inside shopping_cart.py (which is inside the ecommerce folder)

# Single dot means "current package (ecommerce)"
from .payments import process_credit_card

# Double dot means "parent package"
# from ..database import connect
```

Packages as a Whole (__init__.py)

The Initialization File

- Placing an __init__.py file in a directory tells Python to treat that directory as a package.

Exposing the API

- You can use this file to import specific classes from internal modules, making them available directly at the package level.

Simplifying imports

- This hides the complex internal file structure from the user of your package.

Using __init__.py (Code)

Code Snippet:

```
# Inside my_package/__init__.py
# We pull classes from internal files to the package level

from .database import Database
from .network import Server

# The user can now do this in their code:
# from my_package import Database, Server
# Instead of:
# from my_package.database import Database
```

Organizing Code in Modules

Flexibility
- Python does not require one class per file (unlike Java). You can put many related classes in a single module.

Cohesion
- Group classes that work closely together into the same file to keep related logic in one place.

Readability Limits
- If a module becomes thousands of lines long and hard to navigate, it is time to split it into smaller modules within a package.

Multiple Classes in One Module (Code)

Code Snippet:

```
# File: shopping_cart.py

class Item:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

class Cart:
    def __init__(self):
        self.items = []

    def add(self, item: Item):
        self.items.append(item)
```

Handling Circular imports

The Circular Dependency Problem

- Occurs when Module A imports Module B, and Module B tries to import Module A. This crashes Python.

Architectural Fix

- Often, a circular import means your modules are too tightly coupled, and the design should be refactored.

Type Hinting Fix

- If the import is only needed for type hints, Python's typing.TYPE_CHECKING flag can resolve the issue safely.

Fixing Circular Type Hints (Code)

Code Snippet:

```
from typing import TYPE_CHECKING

# This code block ONLY runs during static analysis (like mypy)
# It does NOT run when the program is actually executing.
if TYPE_CHECKING:
    from other_module import ClassB

class ClassA:
    # Use a string for the type hint to avoid runtime errors
    def process(self, obj: 'ClassB'):
        pass
```

Privacy and Encapsulation in Python

No Strict Privacy
- Unlike Java or C++, Python has no private or protected keywords to forcefully hide data.

"We Are All Adults Here"
- Python relies on convention and trust. Developers agree not to alter internal state directly.

Name Mangling
- Prefixing an attribute with two underscores (e.g., __data) modifies its name internally to strongly discourage access, but it is not strictly private.

Third-Party Libraries

The Python Package Index (PyPI)
- A massive repository of free, open-source packages created by the community.

pip Installer
- The standard command-line tool used to download and install packages from PyPI.

Virtual Environments (venv)
- Tools used to create isolated Python environments to prevent package version conflicts between different projects.

Using pip and Virtual Environments (Code)

Code Snippet:

```
# In your terminal: create a virtual environment named 'env'
python -m venv env

# Activate the environment (Windows)
envScriptsactivate
# Activate (Mac/Linux)
source env/bin/activate

# Install a third-party library
pip install requests
# In your Python file:
import requests
```

Case Study – Iris Classification Overview

The Problem

- Classify Iris flowers into species based on physical measurements using k-Nearest Neighbors.

Core Concepts

- Model data samples, split into training/testing sets, and tune hyperparameters.

OOP Goal

- Transition the high-level UML design from Chapter 1 into concrete Python classes.

Logical View – Training Data and Samples

The Sample Class
- Represents a single flower with four physical measurements.

The TrainingData Class
- Container (aggregation) holding two lists of Samples: training and testing.

One-to-Many Relationship
- A single TrainingData instance manages many Sample instances.

Logical View – Hyperparameters

The Hyperparameter Class
- Encapsulates tuning variables like k (number of neighbors).

Tracking Quality
- Stores the success rate (quality) of the algorithm using these parameters.

Collaborating Objects
- TrainingData holds multiple Hyperparameter trials to find the best settings.

Modeling Sample States

Different Sample Types
- Known samples (training) vs. unknown samples (to be classified).

State Changes
- An UnknownSample becomes a ClassifiedSample when the AI predicts its species.

Object Evolution
- Modify an attribute or create a new object? A core design decision.

Tracking Classification States

Optional Attributes
- species and classification may start as None depending on origin.

Type Hinting None
- Use Optional[str] (or str | None in Python 3.10+) for possibly empty attributes.

Consolidated Class
- A single Sample class with optional fields accommodates all states.

The Sample Class Importlementation (Code)

Code Snippet:

```
from typing import Optional

class Sample:
    def __init__(self,
        sepal_length: float, sepal_width: float,
        petal_length: float, petal_width: float,
        species: Optional[str] = None):
        self.sepal_length = sepal_length
        self.species = species
        self.classification: Optional[str] = None
```

State Transition Method (Code)

Code Snippet:

```
class Sample:
    def classify(self, classification: str) -> None:
        """Unknown -> Classified state transition."""
        self.classification = classification


# Usage:
s = Sample(5.1, 3.5, 1.4, 0.2)
s.classify("Iris-setosa")
print(s.classification)  # Iris-setosa
```

Defining Class Responsibilities

The Orchestrator
- TrainingData manages the entire ML workflow.

Loading Data
- Reads raw data and creates Sample objects.

Testing & Tuning
- Runs trials with different Hyperparameter configurations.

Emergent Behavior
- Complex behavior from small focused classes, not one "God Class."

Skeleton of the Orchestrator (Code)

Code Snippet:

```
import datetime
from typing import List

class TrainingData:
    def __init__(self, name: str) -> None:
        self.name = name
        self.uploaded: datetime.datetime
        self.tested: datetime.datetime
        self.training: List['Sample'] = []
        self.testing: List['Sample'] = []
        self.tuning: List['Hyperparameter'] = []
```

Loading the Training Data

Iterables as Inputs
- load() accepts an Iterable, decoupling the class from the file system.

Flexibility
- Works with CSV files, database queries, or web APIs without code changes.

Data Conversion
- Extracts raw dictionaries and instantiates proper Sample objects.

Importlementing the Load Method (Code)

Code Snippet:

```
from typing import Iterable

class TrainingData:
    def load(self, raw_data: Iterable[dict]) -> None:
        for row in raw_data:
            s = Sample(
                float(row["sepal_length"]),
                float(row["sepal_width"]),
                float(row["petal_length"]),
                float(row["petal_width"]),
                species=row["species"])
            self.training.append(s)
        self.uploaded = datetime.datetime.now()
```

Using the TrainingData System (Code)

Code Snippet:

```
import csv

# 1. Create the central object
td = TrainingData("Iris Dataset")

# 2. Open a CSV file and load it
with open("iris.csv") as f:
    reader = csv.DictReader(f)
    td.load(reader)

print(f"Loaded {len(td.training)} samples")
```

Chapter 2 Recall

Modules and Packages

- Organizing code into .py files (modules) and directories (packages) with import statements.

Privacy and Third-Party Libraries

- Convention-based privacy with _ and __ prefixes; leveraging pip and virtual environments for external packages.

Case Study: Iris Classifier

- Translating UML into Python classes: Sample, TrainingData, and Hyperparameter with state management.

Exercises

Build a Class

- Write a simple class representing a real-world object. Include __init__ and type hints.

Create a Package

- Break a single script into two modules inside a package. Practice importing between them.

Explore PyPI

- Install a third-party library using pip inside a virtual environment and import it into your code.

Summary

From Design to Implementation
- We successfully translated object-oriented analysis into functional Python code.

State and Behavior
- We saw how methods act upon the encapsulated state (attributes) of an object using self.

Preparation for Advanced OOP
- Mastering object creation and imports paves the way for understanding inheritance and polymorphism.